



Unit 4 — Loops in Python : while Loop

Learning Module | Beginner-Friendly

LEARNING OBJECTIVES

After completing this module, you will be able to:

- Understand what a while loop is and when to use it in a program
- Write programs using the while loop with various conditions
- Use break, continue, and pass to control loop execution
- Write while loops with else blocks to handle post-loop logic
- Identify and handle infinite loops safely
- Apply nested while loops to solve multi-level problems
- Build complete practical programs using while loop concepts

CLOUD & AI ACADEMY

TABLE OF CONTENTS

1. Introduction to while Loop	3
2. Syntax of while Loop	5
3. Flow of Execution in a while Loop	7
4. Using Conditions in while Loop	9
5. Infinite while Loop	12
6. Using break with while Loop	14
7. Using continue with while Loop	17
8. Using pass in while Loop	20
9. while Loop with else Statement	22
10. Nested while Loops	25

ETIHANS
CLOUD & AI ACADEMY



1. Introduction to while Loop

1.1 What is Repetition in Programming?

Imagine you are asked to print the numbers from 1 to 100. Without loops, you would have to write 100 separate print statements — which is both time-consuming and impractical. Programming loops solve this problem by allowing a block of code to be executed multiple times automatically.

Python provides two types of loops for repetition:

for Loop	while Loop
Used when the number of repetitions is known in advance (e.g., iterate through a list of 10 items).	Used when the number of repetitions is NOT known in advance — the loop keeps running as long as a condition is True.

1.2 What is a while Loop?

A while loop repeatedly executes a block of code as long as a specified condition remains True. Before each iteration (repetition), Python checks the condition. If the condition is True, the loop body runs. Once the condition becomes False, the loop stops and the program continues to the next statement after the loop.

Key Concept

A while loop is a condition-controlled loop. It does not run a fixed number of times — it keeps running as long as its condition evaluates to True.

1.3 Real-World Analogy

Consider an office receptionist who checks whether visitors are still waiting in the queue. The process is:

- While there are visitors in the queue:
 - Call the next visitor.
 - Complete their appointment.
- When the queue is empty, the receptionist closes the desk.

This is exactly how a while loop works in programming — it checks the condition ("is there a visitor?") and keeps executing the loop body until the condition becomes False.

1.4 When Should You Use a while Loop?

Use a while loop when:

- The number of iterations is not known in advance.
- The loop should continue until a user performs a specific action (e.g., enters a correct password).
- The loop should run until a calculation reaches a certain value.
- You are building menu-driven programs that repeat until the user chooses to exit.



2. Syntax of while Loop

2.1 Basic Syntax

Syntax — while Loop

```
while condition:
    # body of the loop
    # (statements to execute repeatedly)
```

Let us break down each part of this syntax:

Component	Description
while	The keyword that starts the loop
condition	A boolean expression that is evaluated before each iteration. The loop runs as long as this is True
:	A mandatory colon after the condition, indicating a block is about to follow
loop body	The indented block of code that executes on each iteration

2.2 The Three Essential Components of a while Loop

Every correctly written while loop must contain three elements. Missing any one of them will either cause an error or result in an infinite loop.

1. Initialisation Set a starting value for the variable used in the condition before the loop begins.	2. Condition The expression checked before each iteration. If False, the loop stops.	3. Update Change the loop variable inside the loop body so the condition eventually becomes False.
---	---	--

2.3 A Minimal while Loop Example

The following example demonstrates all three essential components — initialisation, condition, and update — in the simplest possible form.

Example 2.1 — Counting from 1 to 5

```
# Printing numbers from 1 to 5 using a while loop
```

```
count = 1      # 1. Initialisation: start at 1

while count <= 5: # 2. Condition: keep going while count is <= 5
    print(count) # Loop body: print the current value
    count = count + 1 # 3. Update: increase count by 1

print('Loop has ended.')
```

Output:

```
1
2
3
4
5
Loop has ended.
```

Trace Through

count starts at 1. Each time the loop runs, count increases by 1. When count becomes 6, the condition $6 \leq 5$ is False, so the loop stops and 'Loop has ended.' is printed.

2.4 Indentation is Mandatory

Important Rule

Every statement inside the while loop body must be indented by exactly 4 spaces (or 1 Tab). Python uses indentation to know which statements belong to the loop. Any unindented statement after the loop is treated as outside the loop and runs only once after the loop ends.

Example 2.2 — Indentation Demonstration

```
count = 1
while count <= 3:
    print('Inside loop:', count) # indented — part of loop
    count += 1                  # indented — part of loop
print('Outside loop')          # NOT indented — runs once after loop
```

Output:

```
Inside loop: 1
Inside loop: 2
Inside loop: 3
Outside loop
```



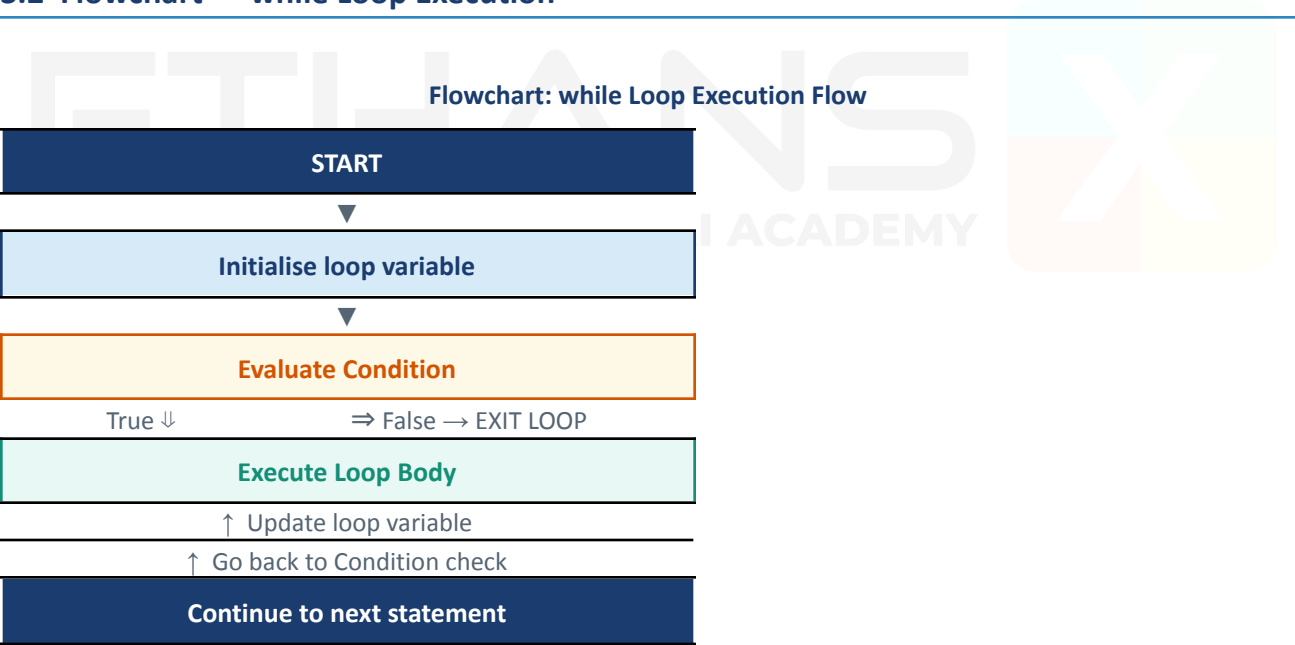
3. Flow of Execution in a while Loop

3.1 Step-by-Step Execution

Understanding how Python executes a while loop step by step is essential before writing complex programs. The execution follows a predictable cycle:

- Step 1:** Python reaches the while statement and evaluates the condition.
- Step 2:** If the condition is True, Python enters the loop body and executes all indented statements.
- Step 3:** After executing the last statement in the loop body, Python goes back to the while condition and evaluates it again.
- Step 4:** Steps 2 and 3 repeat until the condition becomes False.
- Step 5:** When the condition is False, Python exits the loop and continues with the next statement after the loop.

3.2 Flowchart — while Loop Execution



3.3 Tracing a Program Execution

Tracing means following the execution of a program line by line, recording what happens at each step. This is a powerful debugging technique. Let us trace Example 2.1 step by step:

Iteration	count value	Condition (count <= 5)	Action
-----------	-------------	------------------------	--------

Before loop	1	—	Initialised: count = 1
Iteration 1	1	1 <= 5 → True	Print 1, count becomes 2
Iteration 2	2	2 <= 5 → True	Print 2, count becomes 3
Iteration 3	3	3 <= 5 → True	Print 3, count becomes 4
Iteration 4	4	4 <= 5 → True	Print 4, count becomes 5
Iteration 5	5	5 <= 5 → True	Print 5, count becomes 6
Check again	6	6 <= 5 → False	Loop exits

ETHANS
CLOUD & AI ACADEMY



4. Using Conditions in while Loop

4.1 Overview

The condition in a while loop can be any expression that evaluates to True or False. This includes comparison operators, logical operators, or even a variable that holds a boolean value. In this section, we explore different types of conditions and how they control the loop.

4.2 Using Comparison Operators in Conditions

Example 1: Countdown Timer

Write a program that counts down from 10 to 1 and then prints 'Go!'.

Example 4.1 — Countdown Timer

```
# Countdown from 10 to 1
count = 10

while count >= 1:
    print(count)
    count = count - 1

print('Go!')
```

Output:

```
10
9
8
7
6
5
4
3
2
1
Go!
```

Example 2: Sum of First N Natural Numbers

Write a program to calculate the sum of all numbers from 1 to a number entered by the user.

Example 4.2 — Sum of Natural Numbers

```
# Sum of first N natural numbers
```

```
n = int(input('Enter a number: '))
count = 1
total = 0

while count <= n:
    total = total + count
    count = count + 1

print('Sum of first', n, 'numbers is:', total)
```

Output:

```
Enter a number: 5
Sum of first 5 numbers is: 15
```

Example 3: Multiplication Table

Write a program to print the multiplication table of any number entered by the user.

Example 4.3 — Multiplication Table

```
# Multiplication table using while loop
number = int(input('Enter a number for its table: '))
multiplier = 1

print('Multiplication Table of', number)
print('-----')

while multiplier <= 10:
    result = number * multiplier
    print(number, 'x', multiplier, '=', result)
    multiplier = multiplier + 1
```

Output:

```
Enter a number for its table: 7
Multiplication Table of 7
-----
7 x 1 = 7
7 x 2 = 14
7 x 3 = 21
7 x 4 = 28
7 x 5 = 35
7 x 6 = 42
7 x 7 = 49
```

```
7 x 8 = 56
7 x 9 = 63
7 x 10 = 70
```

4.3 Using Logical Operators in Conditions

Example 4: Input Validation Loop

Write a program that keeps asking the user for a number between 1 and 10 until they enter a valid value.

Example 4.4 — Input Validation Loop

```
# Input validation with while loop
number = int(input('Enter a number between 1 and 10: '))

while number < 1 or number > 10:
    print('Invalid input! Please try again.')
    number = int(input('Enter a number between 1 and 10: '))

print('Valid input received:', number)
```

Output:

```
Enter a number between 1 and 10: 15
Invalid input! Please try again.
Enter a number between 1 and 10: -3
Invalid input! Please try again.
Enter a number between 1 and 10: 7
Valid input received: 7
```

Example 5: Password Attempt Limiter

Write a program that allows a user 3 attempts to enter the correct password.

Example 4.5 — Password Attempt Limiter

```
# Password attempt limiter
correct_password = 'python2024'
attempts = 0
max_attempts = 3

while attempts < max_attempts:
    entered = input('Enter password: ')
    attempts = attempts + 1
```

```
if entered == correct_password:
    print('Access granted! Welcome.')
    break
else:
    remaining = max_attempts - attempts
    if remaining > 0:
        print('Wrong password.', remaining, 'attempt(s) remaining.')
    else:
        print('Account locked. Too many failed attempts.')
```

Output:

```
Enter password: hello
Wrong password. 2 attempt(s) remaining.
Enter password: abc
Wrong password. 1 attempt(s) remaining.
Enter password: test
Account locked. Too many failed attempts.
```

ETIHANS
CLOUD & AI ACADEMY



5. Infinite while Loop

5.1 What is an Infinite Loop?

An infinite loop is a loop whose condition never becomes False, causing it to run forever without stopping. This is one of the most common mistakes beginners make when writing while loops.

Warning

An infinite loop will freeze your program. The only way to stop it manually is to press Ctrl + C in the terminal or shell. Always make sure your loop has a condition that will eventually become False, or use a break statement to exit.

5.2 How Does an Infinite Loop Occur?

An infinite loop typically occurs in one of these three situations:

- The loop variable is never updated inside the loop body.
- The condition is always True because it was written incorrectly.
- The update statement moves the variable in the wrong direction.

Example 5.1 — Common Infinite Loop Mistakes (Do Not Run)

```
# INFINITE LOOP — DO NOT RUN (for study purposes only)
# Mistake 1: Forgot to update the loop variable
count = 1
while count <= 5:
    print(count)
    # count is never updated — count stays 1 forever

# Mistake 2: Condition is always True
while True:
    print('This runs forever!')

# Mistake 3: Updating in the wrong direction
count = 1
while count >= 1:
    print(count)
    count = count + 1 # count keeps increasing — never goes below 1
```

How to Stop an Infinite Loop

If your program gets stuck in an infinite loop, press Ctrl + C on Windows/Linux or Cmd + C on Mac. In IDLE, press Ctrl + C or go to Shell > Restart Shell.

5.3 Intentional Infinite Loop with while True

Sometimes, programmers deliberately use an infinite loop using while True: and rely on a break statement to exit the loop at the right moment. This is a valid and common programming pattern, especially for menu-driven programs.

 Example 5.2 — Intentional Infinite Loop with Exit Condition

```
# Intentional infinite loop — exits when user types 'quit'
print('Type any message. Type "quit" to exit.')

while True:
    message = input('You: ')

    if message == 'quit':
        print('Goodbye!')
        break

    print('Echo:', message)
```

Output:

```
Type any message. Type "quit" to exit.
You: Hello
Echo: Hello
You: How are you?
Echo: How are you?
You: quit
Goodbye!
```

6. Using break with while Loop

6.1 What is the break Statement?

The break statement is used to immediately exit a loop, regardless of whether the loop condition is still True. When Python encounters a break statement inside a loop, it jumps out of the loop and continues executing the code after the loop.

Key Concept

break gives you manual control to exit a loop at any point. It is commonly used when you are searching for something and want to stop once you find it, or when a special condition arises that requires stopping the loop immediately.

6.2 Flowchart — break in while Loop

while condition is True:

Execute loop body

if special condition:

break → EXIT loop immediately

Continue to code after loop

6.3 Examples of break

Example 1: Find the First Divisible Number

Write a program to find the first number between 1 and 100 that is divisible by both 7 and 11.

Example 6.1 — Finding First Divisible Number

```
# Find first number divisible by both 7 and 11
number = 1

while number <= 100:
    if number % 7 == 0 and number % 11 == 0:
        print('Found:', number, 'is divisible by both 7 and 11.')
        break
    number = number + 1
```



```
else:  
    print('No such number found in range 1-100.')
```

Output:

Found: 77 is divisible by both 7 and 11.

Example 2: Student Attendance Alert

Write a program that scans a list of student attendance records (entered one by one) and stops immediately when a student with 0% attendance is found.

Example 6.2 — Attendance Alert with break

```
# Scan student attendance — stop at 0% attendance  
print('Enter student attendance (%). Type -1 to stop.')
```



```
student_number = 1
```



```
while True:  
    attendance = float(input('Student ' + str(student_number) + ' attendance: '))  
  
    if attendance == -1:  
        print('Scanning complete.')        break  
  
    if attendance == 0:  
        print('ALERT: Student', student_number, 'has 0% attendance!')        print('Immediate action required. Scan stopped.')        break  
  
    print('Student', student_number, '- Attendance:', attendance, '% [OK]')    student_number += 1
```

Output:

```
Enter student attendance (%). Type -1 to stop.  
Student 1 attendance: 85  
Student 1 - Attendance: 85.0 % [OK]  
Student 2 attendance: 72  
Student 2 - Attendance: 72.0 % [OK]  
Student 3 attendance: 0  
ALERT: Student 3 has 0% attendance!  
Immediate action required. Scan stopped.
```

Example 3: Number Guessing Game with Limited Attempts

Example 6.3 — Guessing Game with break

```
# Number guessing game — max 5 attempts
secret = 42
attempts = 0
max_attempts = 5

while attempts < max_attempts:
    guess = int(input('Guess the number (1-100): '))
    attempts += 1

    if guess == secret:
        print('Correct! You guessed it in', attempts, 'attempt(s).')
        break
    elif guess < secret:
        print('Too low! Try higher.')
    else:
        print('Too high! Try lower.')

if attempts == max_attempts:
    print('Out of attempts! The number was', secret)
```

Output:

```
Guess the number (1-100): 50
Too high! Try lower.
Guess the number (1-100): 25
Too low! Try higher.
Guess the number (1-100): 42
Correct! You guessed it in 3 attempt(s).
```

7. Using continue with while Loop

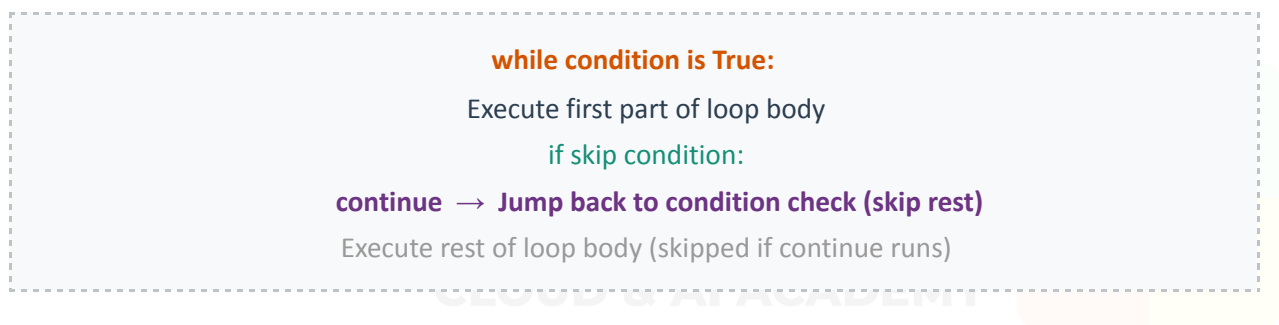
7.1 What is the continue Statement?

The continue statement skips the remaining statements in the current iteration of the loop and jumps back to the condition check for the next iteration. Unlike break (which exits the loop entirely), continue only skips the rest of the current loop cycle.

Key Distinction

break exits the loop entirely. continue only skips the rest of the current iteration and moves on to the next one. The loop itself continues running after a continue statement.

7.2 Flowchart — continue in while Loop



7.3 Examples of continue

Example 1: Skip Even Numbers

Write a program to print only odd numbers from 1 to 10.

Example 7.1 — Print Odd Numbers Using continue

```
# Print only odd numbers from 1 to 10
number = 0

while number < 10:
    number += 1

    if number % 2 == 0: # If the number is even...
        continue      # ...skip the rest and go to next iteration
```

```
print(number)    # This only prints for odd numbers
```

Output:

```
1
3
5
7
9
```

Example 2: Skip Negative Numbers in a List

Write a program that accepts 5 numbers from the user and calculates the sum of only the positive numbers, skipping any negative values.

Example 7.2 — Skip Negatives with continue

```
# Sum of positive numbers — skip negatives
count = 0
total = 0
positive_count = 0

while count < 5:
    num = float(input('Enter number ' + str(count + 1) + ': '))
    count += 1

    if num < 0:
        print('Negative number skipped.')
        continue

    total += num
    positive_count += 1

print('Sum of positive numbers:', total)
print('Count of positive numbers:', positive_count)
```

Output:

```
Enter number 1: 10
Enter number 2: -5
Negative number skipped.
Enter number 3: 20
Enter number 4: -3
Negative number skipped.
Enter number 5: 15
```

```
Sum of positive numbers: 45.0  
Count of positive numbers: 3
```

Example 3: Generate Employee Report (Skip Absent Employees)

Write a program that processes salary for 5 employees. If an employee is marked absent, skip the salary calculation for that employee.

Example 7.3 — Employee Payroll with continue

```
# Employee salary report — skip absent employees  
emp_number = 1  
total_payout = 0  
  
while emp_number <= 5:  
    name = input('Employee ' + str(emp_number) + ' name: ')  
    status = input('Status (Present/Absent): ').strip()  
  
    if status == 'Absent':  
        print(name, '- Absent. Salary not processed.\n')  
        emp_number += 1  
        continue  
  
    salary = float(input('Enter salary for ' + name + ': '))  
    total_payout += salary  
    print(name, '- Salary processed: Rs.', salary, '\n')  
    emp_number += 1  
  
print('Total Payout for the Month: Rs.', total_payout)
```

Output:

```
Employee 1 name: Arjun  
Status (Present/Absent): Present  
Enter salary for Arjun: 35000  
Arjun - Salary processed: Rs. 35000.0  
  
Employee 2 name: Priya  
Status (Present/Absent): Absent  
Priya - Absent. Salary not processed.  
  
Total Payout for the Month: Rs. 35000.0
```



8. Using pass in while Loop

8.1 What is the pass Statement?

The pass statement is a placeholder in Python. It does absolutely nothing when executed — it is simply there to satisfy Python's syntax requirement that a code block cannot be empty.

In Python, every if, while, for, or function block must contain at least one statement. If you have not yet written the code for a block but want to avoid a syntax error, you can use pass as a temporary placeholder.

When is pass Used?

pass is mainly used during development when you want to create the structure of your program first and fill in the logic later. It is also used when a condition must be handled syntactically but no action is needed.

8.2 Syntax of pass

Syntax — pass in while Loop

```
while condition:
    pass # Do nothing — placeholder for future code
```

Important Distinction

pass, break, and continue are all used inside loops but do very different things: pass does nothing and lets the loop continue normally. break exits the loop immediately. continue skips the rest of the current iteration.

Statement	Effect on Loop	Use Case
pass	Does nothing — loop continues normally	Placeholder for future code
break	Exits the loop immediately	Stop the loop early
continue	Skips rest of current iteration, goes to next	Skip specific iterations

8.3 Examples of pass

Example 1: Skeleton Program — Future Implementation

You are building a monitoring system. You have designed the loop structure but have not yet written the monitoring logic. Use pass to create a valid program skeleton.

Example 8.1 — Using pass as a Placeholder

```
# Server monitoring system — skeleton
monitoring = True
check_count = 0

while monitoring and check_count < 3:
    # TODO: Add server health check logic here
    pass
    check_count += 1

print('Monitoring cycle complete.')
```

Output:

```
Monitoring cycle complete.
```

Example 2: Ignoring Specific Iterations Without Skipping

Write a loop that processes numbers 1 to 5, but for the number 3, no special processing is needed — simply move on.

Example 8.2 — pass vs Skipping Logic

```
# Process numbers 1 to 5
# Number 3 needs no special processing
number = 1

while number <= 5:
    if number == 3:
        pass # No action needed for 3, continue normally
    else:
        print('Processing number:', number)
    number += 1
```

Output:

```
Processing number: 1
Processing number: 2
Processing number: 4
Processing number: 5
```

Notice that number 3 was visited by the loop (the loop did not skip it), but no output was produced for it because `pass` did nothing. This is different from `continue` — with `continue`, the loop would have jumped to

the next iteration at that point. With pass, execution simply continued to the else block for other numbers (but there is no else for 3, so nothing prints).



9. while Loop with else Statement

9.1 What is the while-else Statement?

Python has a unique feature that most other programming languages do not have: the else clause on a while loop. The else block associated with a while loop executes only when the loop condition becomes False naturally — that is, when the loop completes all its iterations without being interrupted by a break statement.

Key Rule

The else block runs when the while condition becomes False (normal completion). If the loop is terminated by a break statement, the else block is SKIPPED entirely. This makes while-else very useful for search operations.

9.2 Syntax of while-else

Syntax — while-else Statement

```
while condition:
```

```
    # loop body
```

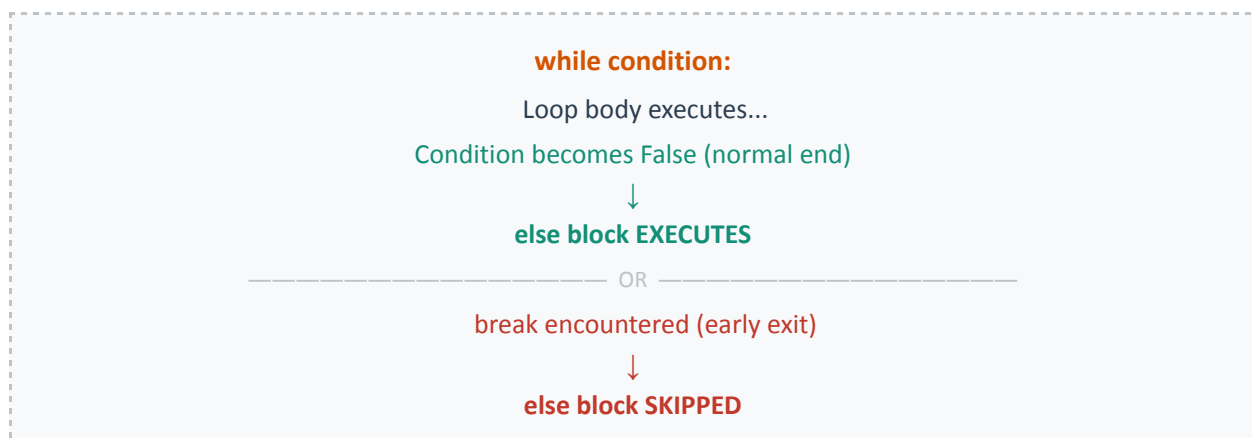
```
    # if break is reached, else is SKIPPED
```

```
else:
```

```
    # executes ONLY if the loop ended naturally (condition became False)
```

```
    # does NOT execute if loop was exited via break
```

9.3 Flowchart — while-else



9.4 Examples of while-else

Example 1: Search for a Number

Write a program to search for a specific number in a range. If found, report it. If the entire range is checked and the number is not found, report that too.

Example 9.1 — Search with while-else

```
# Search for a number in a range
target = 37
number = 1

while number <= 50:
    if number == target:
        print('Number', target, 'found at position', number)
        break
    number += 1
else:
    print('Number', target, 'was NOT found in the range 1 to 50.')
```

Output:

Number 37 found at position 37

Now let us test with a number not in the range:

Example 9.2 — while-else When Target Not Found

```
# Searching for 75 in range 1-50
target = 75
number = 1

while number <= 50:
    if number == target:
        print('Number', target, 'found!')
        break
    number += 1
else:
    print('Number', target, 'was NOT found in the range 1 to 50.')
```

Output:

Number 75 was NOT found in the range 1 to 50.

Example 2: Login with Attempts — Report Result

Example 9.3 — Login System with while-else

```
# Login system with attempt tracking
correct_password = 'secure123'
attempts = 0
max_attempts = 3

while attempts < max_attempts:
    password = input('Enter password: ')
    attempts += 1

    if password == correct_password:
        print('Login successful! Welcome.')
        break
    else:
        print('Incorrect. Attempts left:', max_attempts - attempts)
else:
    # This runs only if all 3 attempts were used up (no break occurred)
    print('\nAll attempts exhausted. Account has been locked.')
    print('Please contact support to unlock your account.')
```

Output:

```
Enter password: wrongpass
Incorrect. Attempts left: 2
Enter password: test123
Incorrect. Attempts left: 1
Enter password: hello
Incorrect. Attempts left: 0
```

```
All attempts exhausted. Account has been locked.
Please contact support to unlock your account.
```

10. Nested while Loops

10.1 What are Nested while Loops?

A nested while loop is a while loop that is written inside another while loop. The outer loop controls the major repetition, and for each iteration of the outer loop, the inner loop runs through its complete cycle from start to finish.

Syntax — Nested while Loop

```
while outer_condition:
    # outer loop body
    while inner_condition:
        # inner loop body
        # inner update
    # outer update
```

Execution Order

For every single iteration of the outer loop, the inner loop runs completely (all its iterations) from beginning to end. The inner loop variable must be re-initialised at the start of each outer loop iteration.

10.2 Trace Through a Nested Loop

Let us understand the execution order with a simple example — printing a 3x3 pattern:

Example 10.1 — Nested Loop Trace

```
# Nested while loop — 3x3 pattern
i = 1
while i <= 3:    # outer loop runs 3 times
    j = 1
    while j <= 3: # inner loop runs 3 times for EACH outer iteration
        print('i =', i, ', j =', j)
        j += 1
    i += 1
```

Output:

```
i = 1 , j = 1
i = 1 , j = 2
i = 1 , j = 3
i = 2 , j = 1
```

```
i = 2 , j = 2
i = 2 , j = 3
i = 3 , j = 1
i = 3 , j = 2
i = 3 , j = 3
```

Outer (i)	Inner (j)	Output
1	1, 2, 3	i=1,j=1 i=1,j=2 i=1,j=3
2	1, 2, 3	i=2,j=1 i=2,j=2 i=2,j=3
3	1, 2, 3	i=3,j=1 i=3,j=2 i=3,j=3

10.3 Practical Examples of Nested while Loops

Example 1: Printing a Star Pattern

Print a right-angled triangle pattern of stars where each row has an increasing number of stars.

Example 10.2 — Star Triangle Pattern

```
# Star triangle pattern
rows = int(input('Enter number of rows: '))
i = 1

while i <= rows:
    j = 1
    while j <= i: # inner loop runs i times
        print('*', end=' ')
        j += 1
    print()      # move to the next line after each row
    i += 1
```

Output:

```
Enter number of rows: 5
*
* *
* * *
* * * *
* * * * *
```

Example 2: Multiplication Table for Multiple Numbers

Print multiplication tables for all numbers from 1 to 3.

Example 10.3 — Multiple Multiplication Tables

```
# Multiplication tables for 1 to 3
number = 1

while number <= 3:
    print('--- Table of', number, '---')
    multiplier = 1

    while multiplier <= 5:
        print(number, 'x', multiplier, '=', number * multiplier)
        multiplier += 1

    print() # blank line between tables
    number += 1
```

Output:

```
--- Table of 1 ---
1 x 1 = 1
1 x 2 = 2
1 x 3 = 3
1 x 4 = 4
1 x 5 = 5

--- Table of 2 ---
2 x 1 = 2
2 x 2 = 4
2 x 3 = 6
2 x 4 = 8
2 x 5 = 10

--- Table of 3 ---
3 x 1 = 3
3 x 2 = 6
3 x 3 = 9
3 x 4 = 12
3 x 5 = 15
```

Example 3: Student Marks Entry System

Write a program for a teacher to enter marks for multiple students, each having multiple subjects. The outer loop iterates over students; the inner loop iterates over subjects for each student.

Example 10.4 — Marks Entry System

```
# Student marks entry system
num_students = int(input('Enter number of students: '))
num_subjects = int(input('Enter number of subjects: '))

student = 1
while student <= num_students:
    name = input('\nEnter name of Student ' + str(student) + ': ')
    total = 0
    subject = 1

    while subject <= num_subjects:
        marks = float(input(' Subject ' + str(subject) + ' marks: '))
        total += marks
        subject += 1

    average = total / num_subjects
    print(' Total:', total, '| Average:', round(average, 2))
    student += 1
```

Output:

```
Enter number of students: 2
Enter number of subjects: 3

Enter name of Student 1: Ananya
Subject 1 marks: 78
Subject 2 marks: 85
Subject 3 marks: 92
Total: 255.0 | Average: 85.0

Enter name of Student 2: Rohan
Subject 1 marks: 65
Subject 2 marks: 70
Subject 3 marks: 60
Total: 195.0 | Average: 65.0
```


MODULE SUMMARY

The table below summarises all while loop concepts covered in this module.

Concept	Purpose	Key Syntax
while loop	Repeats code while a condition is True	while condition:
Infinite loop	Runs forever — use while True: with break to control	while True:
break	Exits the loop immediately	break
continue	Skips the rest of the current iteration	continue
pass	Does nothing — placeholder for future code	pass
while-else	else runs only if the loop ended naturally (no break)	while ... else:
Nested while	A while loop inside another while loop	while inside while

What Comes Next?

In the next module, you will learn about the for Loop in Python — which is used when the number of iterations is known in advance. You will also learn about the range() function and how to iterate through lists, strings, and other sequences. Together, while and for loops form the complete repetition toolkit in Python.